Kabul Polytechnic University
Computer Science Faculty
Information System Department

Database II

Lecture 2: Enhanced Entity Relationship (EER) Modeling

18 August – 2026

Lectures & other related materials are available here: *https://github.com/mujtabaSultani01/Database-II*

Contents

- EER stands for Enhanced ER or Extended ER
- EER Model Concepts
  - Includes all modeling concepts of basic ER
  - Additional concepts:
    - ✓ Subclasses/superclasses
    - ✓ Specialization/generalization
    - ✓ Attribute and relationship inheritance
  - Constraints on Specialization/Generalization

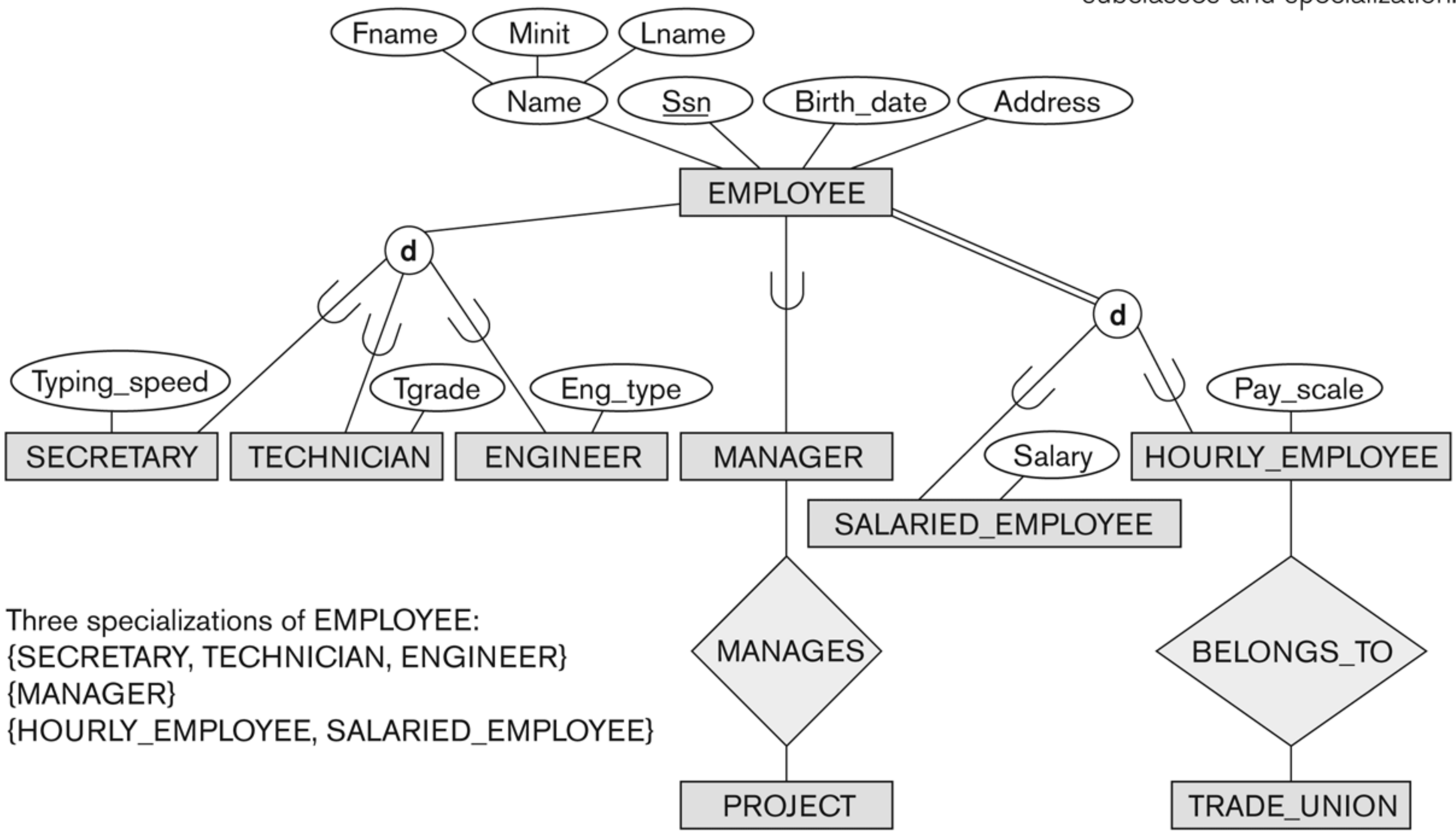
Subclasses and Superclasses (1)

- An entity type may have additional meaningful subgroupings of its attributes.
  - Example: EMPLOYEE may be further grouped into:
    - SECRETARY, ENGINEER, TECHNICIAN, …
      - Based on the EMPLOYEE's Job
    - MANAGER
      - EMPLOYEEs who are managers
    - SALARIED_EMPLOYEE, HOURLY_EMPLOYEE
      - Based on the EMPLOYEE's method of pay

- EER diagrams or extend ER diagrams to represent these additional subgroupings, called *subclasses* or *subtypes*.

Subclasses and Superclasses (2)

Figure 4.1
EER diagram notation to represent subclasses and specialization.



Subclasses and Superclasses (3)

- Each of these **subgroupings** is a subset of EMPLOYEE entity.
- Each is called a **subclass** of EMPLOYEE .
- EMPLOYEE is the **superclass** for each of these subclasses.

- These are called superclass/subclass relationships:
  - EMPLOYEE/SECRETARY
  - EMPLOYEE/TECHNICIAN
  - EMPLOYEE/MANAGER
  - …

Subclasses and Superclasses (4)

- These are also called IS-A relationships
  - SECRETARY IS-A EMPLOYEE, TECHNICIAN IS-A EMPLOYEE, ….

- **Note:** Subclass represents the same real-world entity as some member of the superclass:

  - The subclass member is the same entity in a ***distinct specific role***.
  - To exist in the database as a member of a subclass, the entity **must also exist in the superclass**.
  - A superclass member can belong to multiple subclasses.

Subclasses and Superclasses (5)

- Examples:
  - A salaried employee who is also an engineer belongs to the two subclasses:
    - ENGINEER, and
    - SALARIED_EMPLOYEE
  - A salaried employee who is also an engineering manager belongs to the three subclasses:
    - MANAGER,
    - ENGINEER, and
    - SALARIED_EMPLOYEE
- It is not necessary that every entity in a superclass be a member of some subclass.

Attribute Inheritance in Superclass / Subclass Relationships

- An entity that is member of a subclass *inherits:*
  - All **attributes** of the entity as a member of the superclass
  - All **relationships** of the entity as a member of the superclass

- Example:
  - In the previous slide, SECRETARY (as well as TECHNICIAN and ENGINEER) inherit the attributes Name, SSN, …, from EMPLOYEE
  - Every SECRETARY entity will have values for the inherited attributes.

Specialization (1)

- Specialization is the process of defining a set of subclasses of a superclass.
- The set of subclasses is based upon some distinguishing characteristics of the entities in the superclass.
  - **Example:** {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of EMPLOYEE based upon *job type*.
    - ✓ May have several specializations of the same superclass

Specialization (2)

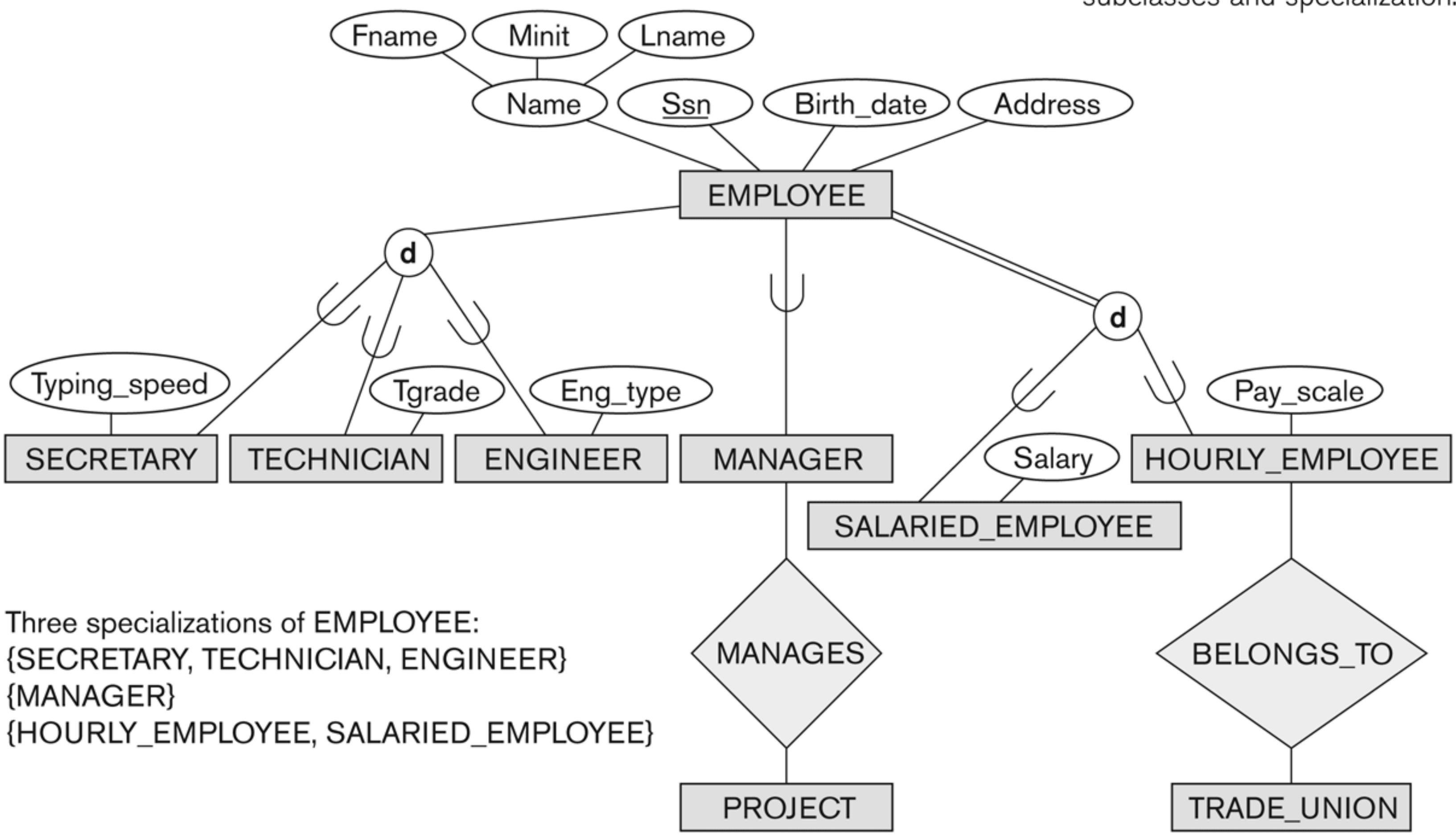
- **Example:** Another specialization of EMPLOYEE based on *method of pay* is {SALARIED_EMPLOYEE, HOURLY_EMPLOYEE}.

  - Superclass/subclass relationships & specialization are shown in EER diagrams.
  - Attributes of a subclass are called *Specific* or *local* attributes.
    - ✓ For example, the attribute TypingSpeed of SECRETARY

  - The subclass can also participate in specific relationship types.
    - ✓ For example, a relationship BELONGS_TO of HOURLY_EMPLOYEE

Specialization (3)

Figure 4.1
EER diagram notation to represent subclasses and specialization.



Three specializations of EMPLOYEE:
{SECRETARY, TECHNICIAN, ENGINEER}
{MANAGER}
{HOURLY_EMPLOYEE, SALARIED_EMPLOYEE}

Generalization (1)

- Generalization is the **reverse of the specialization** process.
- Several classes with common features are generalized into a superclass;
  - Original classes become its subclasses

- Example: CAR, TRUCK generalized into VEHICLE;
  - Both CAR, TRUCK become subclasses of the superclass VEHICLE.
  - We can view {CAR, TRUCK} as a specialization of VEHICLE.
  - Alternatively, we can view VEHICLE as a generalization of CAR and TRUCK.

Generalization (2)

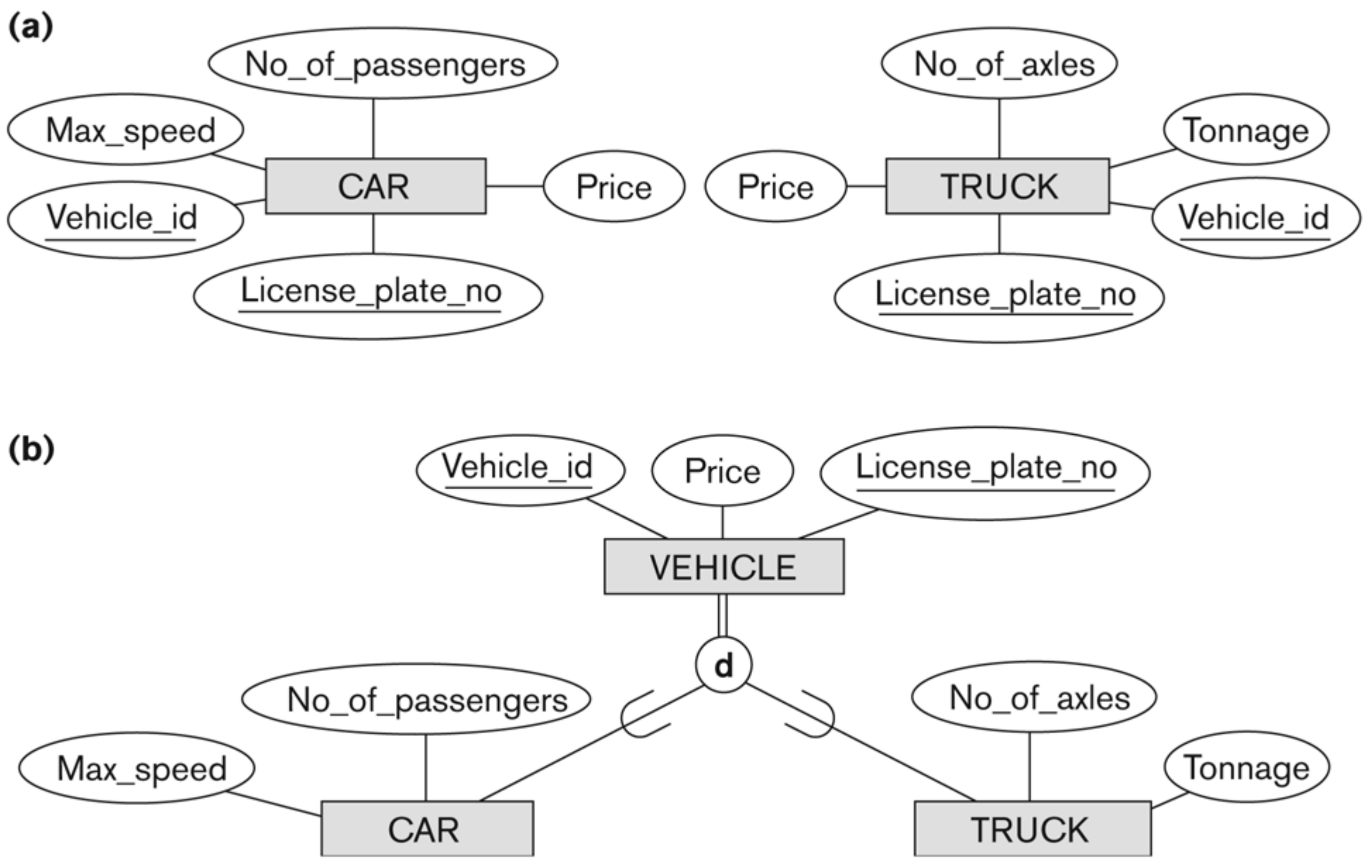


Figure 4.3
Generalization. (a) Two entity types, CAR and TRUCK.
(b) Generalizing CAR and TRUCK into the superclass VEHICLE.

Types of Specialization

- **Predicate-defined (or condition-defined):** Uses a **logical condition** or **range** on one or more attributes. The system automatically decides the subclass. E.g. **Age**, **exp, …**

  - **Child** if age < 18
  - **Adult** if 18 ≤ age ≤ 60
  - **Senior** if age > 60

  - **Junior** if experience < 5
  - **Senior** if experience ≥ 5

Types of Specialization

- **Attribute-defined:** Uses the **specific value** of a single attribute. Membership is directly tied to that attribute's value. E.g. Vehicle **type**, student **program.**

  - **Car** if type = 'Car'
  - **Bike** if type = 'Bike'
  - **Truck** if type = 'Truck'

  - **Undergraduate** if program = 'BSc'
  - **Graduate** if program = 'MSc'
  - **PhD** if program = 'PhD'

Types of Specialization

- **User-defined:** membership is decided manually by the user for each entity.

  - **Entity**: Person
  - **Subclasses**: Customer and Supplier

  - **Entity**: Employee
  - **Subclasses**: TeamLeader or Mentor

  - **Entity**: Product
  - **Subclasses**: FeaturedProduct or StandardProduct

Constraints on Specialization & Generalization (1)

- If we can determine exactly those entities that will become members of each subclass by a condition, the subclasses are called **predicate-defined** (or **condition-defined) subclasses**.
  - **Condition is a constraint** that determines subclass members.
  - **Display a predicate-defined subclass by writing the predicate condition** next to the line attaching the subclass to its superclass.

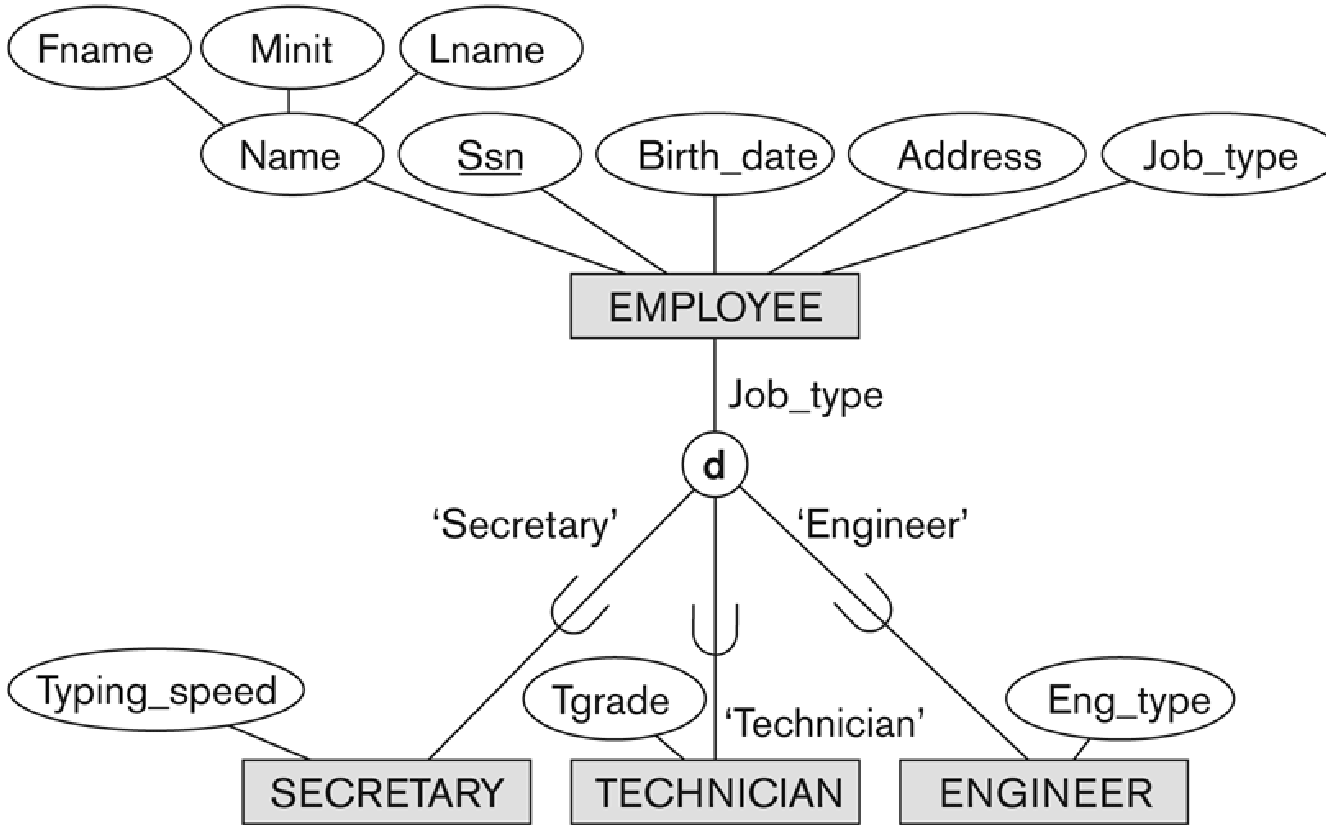
Constraints on Specialization & Generalization (2)

- If all subclasses in a specialization have membership condition on same attribute of the superclass, specialization is called an **attribute-defined specialization.**
  - Attribute is called the defining attribute of the specialization
  - **Example:** JobType is the defining attribute of the specialization {SECRETARY, TECHNICIAN, ENGINEER} of EMPLOYEE

- If **no condition** determines membership, the subclass is called **user-defined.**
  - Membership in a subclass is determined by the **database users** by **applying an operation** to add an entity to the subclass.

Constraints on Specialization and Generalization (3)

Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.



Constraints on Specialization and Generalization (4)

- Two basic constraints can apply to a specialization / generalization:

  1) Disjointness Constraint
  2) Completeness Constraint

Constraints on Specialization and Generalization (5)

- Disjointness Constraint:
  - Specifies that the subclasses of the specialization must be *disjoint*:
    - ✓ An entity can be a member of at **most one of the subclasses** of the specialization.
    - ✓ Example: An employee **cannot be both salaried and hourly**.
  - Specified by ***d*** in EER diagram

  - If not disjoint, specialization is called ***overlapping***.
    - ✓ That is the same entity may be a **member of more than one subclass** of the specialization
  - Specified by **o** in EER diagram

Constraints on Specialization and Generalization (6)

- Completeness Constraint:
  - ***Total:*** Specifies that every entity in the superclass must belong to **at least one subclass**.
  - Shown in EER diagrams by a ***double line.***

  - **Example:**

    Imagine a **Person** entity with subclasses **Employee** and **Student**. If **total completeness** is applied, then every **Person must be either** an Employee or a Student. There **cannot** be a Person who is **neither**.

Constraints on Specialization and Generalization (7)

- Completeness Constraint:
  - ***Partial:*** Allows an instance **not to belong to any of the subclasses**.
  - Shown in EER diagrams by a **single line**.

  - **Example:**
    - ✓ With the **Person** entity, if **partial completeness** is used, there might be **Persons** who are **neither Employees nor Students**. **For example**, a **Retired Person** who does not fit into either subclass.

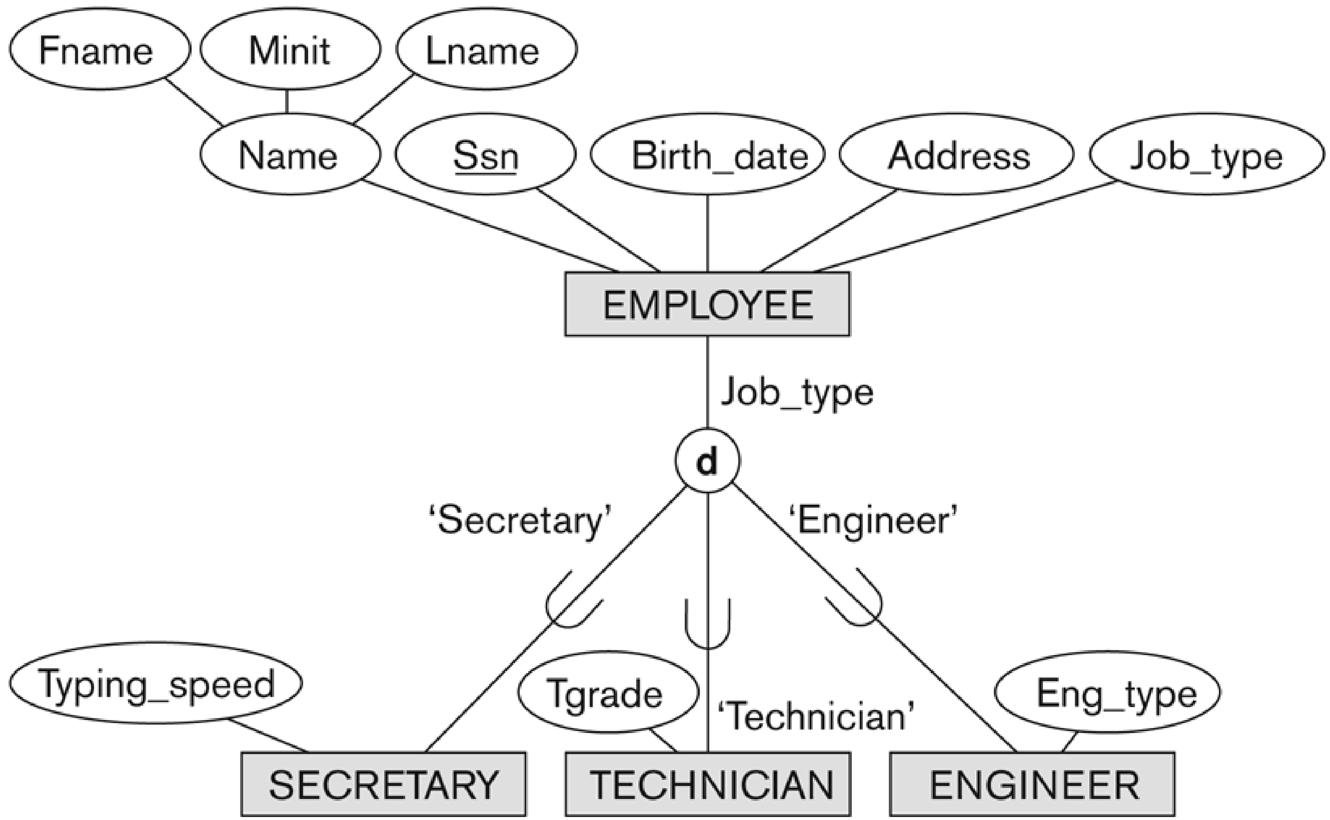
Constraints on Specialization and Generalization (8)

- Hence, we have four types of specialization/generalization:
  1) **Disjoint & total** (every person is either Employee or Student, but not both)
  2) **Disjoint & partial** (some employees may belong to neither subclass)
  3) **Overlapping & total** (every employee is at least one, but could be both)
  4) **Overlapping & partial** (some employees may belong to none or more than one subclass)

- **Note:** Generalization usually is total because the superclass is derived from the subclasses.

Example of disjoint partial Specialization

Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.



Example of overlapping total Specialization

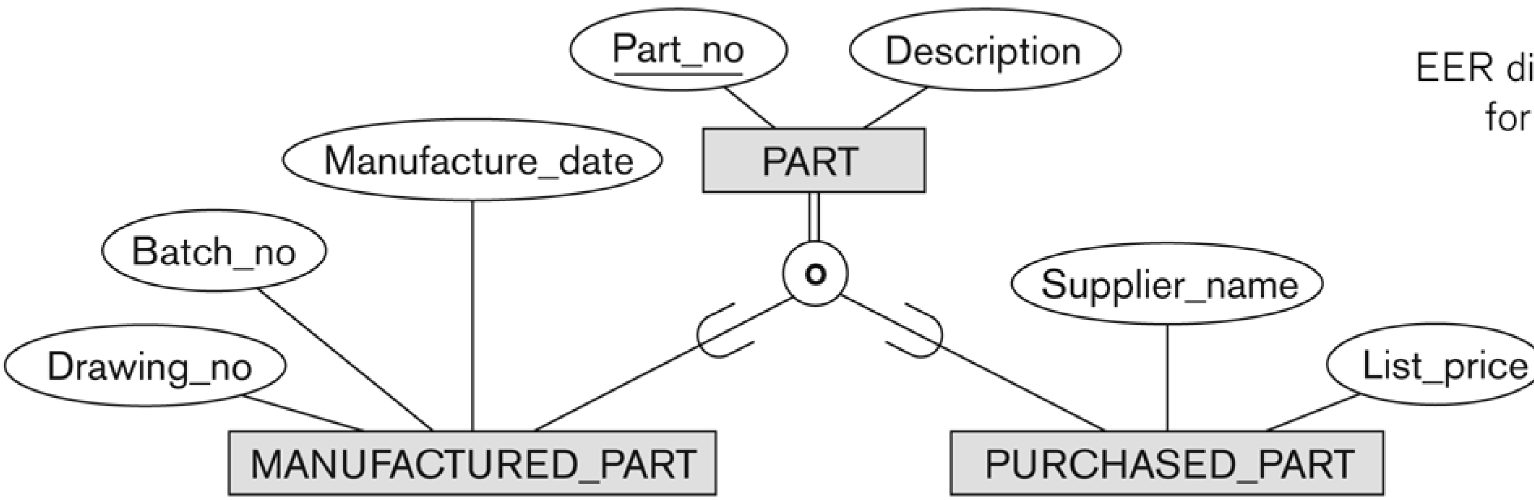


Figure 4.5
EER diagram notation for an overlapping (nondisjoint) specialization.

Conclusion

- Introduced the EER model concepts
  - Class/subclass relationships
  - Specialization and generalization
  - Inheritance

Questions …?